W1

POLYMORPHISM

Data Structures & Object-Oriented Programming



Table of Contents

- ปัญหาของ isinstance() Chain
- Abstract Base Classes (ABC)
- Polymorphism
- @abstractmethod
- Duck Typing
- Common Mistakes

ปัญหาของ isinstance() Chain

- จากสัปดาห์ที่ 3 เราสร้าง Dog, Cat, และ Animal โดยใช้ Inheritance ได้แล้ว
- สมมติว่าต้องการเขียนโปรแกรมคำนวณพื้นที่รูปทรงหลายชนิด ถ้าไม่รู้จัก Polymorphism อาจเขียนได้ดังนี้:

```python
def calculate_total_area(shapes: list) -> float:
    total = 0.0
    for shape in shapes:
        if isinstance(shape, Circle):
            total += 3.14159 * shape.radius ** 2
        elif isinstance(shape, Rectangle):
            total += shape.width * shape.height
        elif isinstance(shape, Triangle):
            total += 0.5 * shape.base * shape.height
        # ถ้าเพิ่ม Hexagon, Pentagon, Ellipse → ต้องแก้ function นี้ทุกครั้ง!
    return total
```

- ทุกครั้งที่เพิ่มรูปทรงใหม่ต้องแก้ function นี้ด้วย
- logic การคำนวณกระจัดกระจายอยู่นอก class ทั้งที่ควรอยู่ใน class
- ถ้า Circle เปลี่ยนชื่อ attribute จาก radius เป็น r code นี้พัง
- ยิ่งมีรูปทรงมากขึ้น elif chain ยาวขึ้นเรื่อยๆ ไม่มีที่สิ้นสุด

Polymorphism แก้ปัญหาทั้งหมดโดยให้แต่ละ class รับผิดชอบ logic ของตัวเอง และ function ภายนอกเพียงแค่ "ขอ" ให้ object ทำสิ่งที่มันทำได้

Polymorphism

Polymorphism

- Polymorphism มาจากภาษากรีก แปลว่า "หลายรูปแบบ"
- ในบริบท OOP หมายความว่า เรียก method ชื่อเดียวกันบน object ต่างชนิด แต่ได้ผลลัพธ์ที่เหมาะสมกับแต่ละชนิด โดยอัตโนมัติ

```
shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 4, 5)]

for shape in shapes:
    shape.area()   ← เรียก method เดียวกัน
         |
         ├── Circle.area()    → π × 5² = 78.54
         ├── Rectangle.area() → 4 × 6 = 24.00
         └── Triangle.area()  → 6.00

Python เลือก implementation ที่ถูกต้องให้อัตโนมัติ
```

กุญแจสำคัญคือ calculate_total_area ที่เขียนใหม่ไม่ต้องรู้เลยว่ากำลังทำงานกับรูปทรงชนิดไหนเป็น type อะไร

Polymorphism — function ไม่รู้และไม่ต้องรู้ว่า shape เป็น

```
def calculate_total_area(shapes: list) -> float:
    return sum(shape.area() for shape in shapes)
```

← ไม่มี isinstance() เลย

หลักการเขียนโปรแกรมเชิง Polymorphism

- Polymorphism หมายถึง การออกแบบให้โค้ดส่วนที่ "เรียกใช้งาน" ไม่จำเป็นต้องรู้ว่ากำลังจัดการกับ object ประเภทใด อยู่ ขอเพียง object นั้นมี method ที่ต้องการก็เพียงพอ
- เป้าหมายคือให้โค้ดส่วนกลางมีความเสถียรและไม่ต้องเปลี่ยนแปลง แม้จะมีการเพิ่ม class ใหม่เข้ามาในระบบในอนาคต

- **ขั้นตอนที่ 1 — ระบุ "Interface ร่วม" ให้ชัดเจนก่อน**
  - "สิ่งที่ต้องการให้ทุก class ทำได้เหมือนกันคืออะไร?"
    - คำตอบนั้น คือ interface ร่วมที่จะเป็นจุดศูนย์กลางของ Polymorphism
  - เช่น หากต้องการจัดการรูปทรงหลายประเภท interface ร่วมที่สมเหตุสมผลคือ
    - ทุกรูปทรงต้องคำนวณพื้นที่ได้ (area()) และคำนวณเส้นรอบรูปได้ (perimeter())
    - โดยไม่สนใจว่าแต่ละรูปทรงจะคำนวณอย่างไรในรายละเอียด

หลักการเขียนโปรแกรมเชิง Polymorphism

- **ขั้นตอนที่ 2 — กำหนด Abstract Class เพื่อบังคับ Interface**
  - กำหนด Abstract Class เพื่อทำหน้าที่เป็น "สัญญา" ที่บังคับให้ทุก class ที่ inherit ไปต้อง implement method เหล่านั้นอย่างครบถ้วน
  - หาก class ใดลืม implement Python จะแจ้ง error ทันทีตอนสร้าง object

```python
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self) -> float:
        pass

    @abstractmethod
    def perimeter(self) -> float:
        pass
```

ประกาศ Abstract Class
บังคับว่าทุก subclass ต้องมี method นี้

บังคับว่าทุก subclass ต้องมี method นี้

Abstract Class ไม่สามารถสร้าง object ได้โดยตรง เพราะมันคือ blueprint ที่ยังไม่สมบูรณ์ ไม่มีตัวตนในโลกจริง

หลักการเขียนโปรแกรมเชิง Polymorphism

- **ขั้นตอนที่ 3 — Implement แต่ละ Class ด้วยรายละเอียดของตัวเอง**
  - สร้าง **concrete class** แต่ละตัวโดย inherit จาก Abstract Class และ implement method ที่กำหนดไว้ด้วย logic ที่เหมาะสมกับตัวเอง
  - แต่ละ class มีอิสระในการคำนวณตามวิธีของตัวเองอย่างสมบูรณ์

```python
import math

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return math.pi * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * math.pi * self.radius
```

```python
class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

    def perimeter(self) -> float:
        return 2 * (self.width + self.height)
```

```python
class Triangle(Shape):
    def __init__(self, a: float, b: float, c: float):
        self.a = a
        self.b = b
        self.c = c

    def area(self) -> float:
        s = (self.a + self.b + self.c) / 2
        return math.sqrt(s * (s-self.a) * (s-self.b) * (s-self.c))

    def perimeter(self) -> float:
        return self.a + self.b + self.c
```

หลักการเขียนโปรแกรมเชิง Polymorphism

- **ขั้นตอนที่ 4 — เขียนโค้ดส่วนกลางที่ไม่สนใจประเภท**
  - โค้ดส่วนกลางสามารถทำงานกับทุก class ได้ผ่าน interface ร่วม โดยไม่มี isinstance() หรือ if/elif ใดๆ ทั้งสิ้น

```
def print_shape_info(shape: Shape) -> None:
    print(f"พื้นที่:        {shape.area():.2f}")
    print(f"เส้นรอบรูป:  {shape.perimeter():.2f}")

shapes = [
    Circle(5),
    Rectangle(4, 6),
    Triangle(3, 4, 5),
]

for shape in shapes:
    print_shape_info(shape)    # ไม่รู้ว่าเป็น class อะไร แต่เรียกได้เลย
```

- สังเกตว่า print_shape_info() ไม่รู้เลยว่ากำลังรับ Circle, Rectangle หรือ Triangle อยู่ และไม่จำเป็นต้องรู้
- รู้เพียงอย่างเดียวว่า object ที่รับเข้ามาต้องมี area() และ perimeter() ซึ่ง Abstract Class รับประกันไว้แล้ว

Duck Typing

Duck Typing

- Python ใช้หลักการที่เรียกว่า Duck Typing: "ถ้ามันเดินแบบเป็ดและร้องแบบเป็ด มันก็คือเป็ด"
- Python ไม่ตรวจสอบว่า object เป็น type อะไร แต่ตรวจสอบว่ามี method ที่ต้องการหรือไม่

```python
import math

# SquareTile ไม่ได้ inherit จาก Shape class ใดๆ เลย
class SquareTile:
    def __init__(self, side: float):
        self.side = side

    def area(self) -> float:
        return self.side ** 2

    def perimeter(self) -> float:
        return 4 * self.side

    def describe(self) -> str:
        return f"SquareTile: พื้นที่ = {self.area():.2f} | เส้นรอบวง = {self.perimeter():.2f}"
```

ทำงานร่วมกับ calculate_total_area ได้ทันที — Python ไม่ตรวจ type

```python
tile = SquareTile(3.0)
shapes = [Circle(5.0), Rectangle(4.0, 6.0), tile]
print(calculate_total_area(shapes))

print(isinstance(tile, Shape))
```

78.54 + 24.00 + 9.00 = 111.54

False — tile ไม่ใช่ Shape

แต่ calculate_total_area ยังทำงานได้ เพราะ tile มี .area()

Duck Typing ทำให้ Python มี flexibility สูง
แต่ข้อเสียคือถ้า object ไม่มี method ที่ต้องการ จะได้ AttributeError ตอน runtime ไม่ใช่ตอน compile

Abstract Base Classes (ABC)

Abstract Base Classes (ABC)

- Duck Typing ให้ความยืดหยุ่น แต่ไม่มีอะไรป้องกันนักพัฒนาที่สร้าง subclass แล้วลืม implement method สำคัญ
- Abstract Base Class (ABC) แก้ปัญหานี้โดยกำหนด สัญญา (contract) ที่ทุก subclass ต้องปฏิบัติตาม

```
from abc import ABC, abstractmethod

class Shape(ABC):   # ← inherit จาก ABC
    ...
```

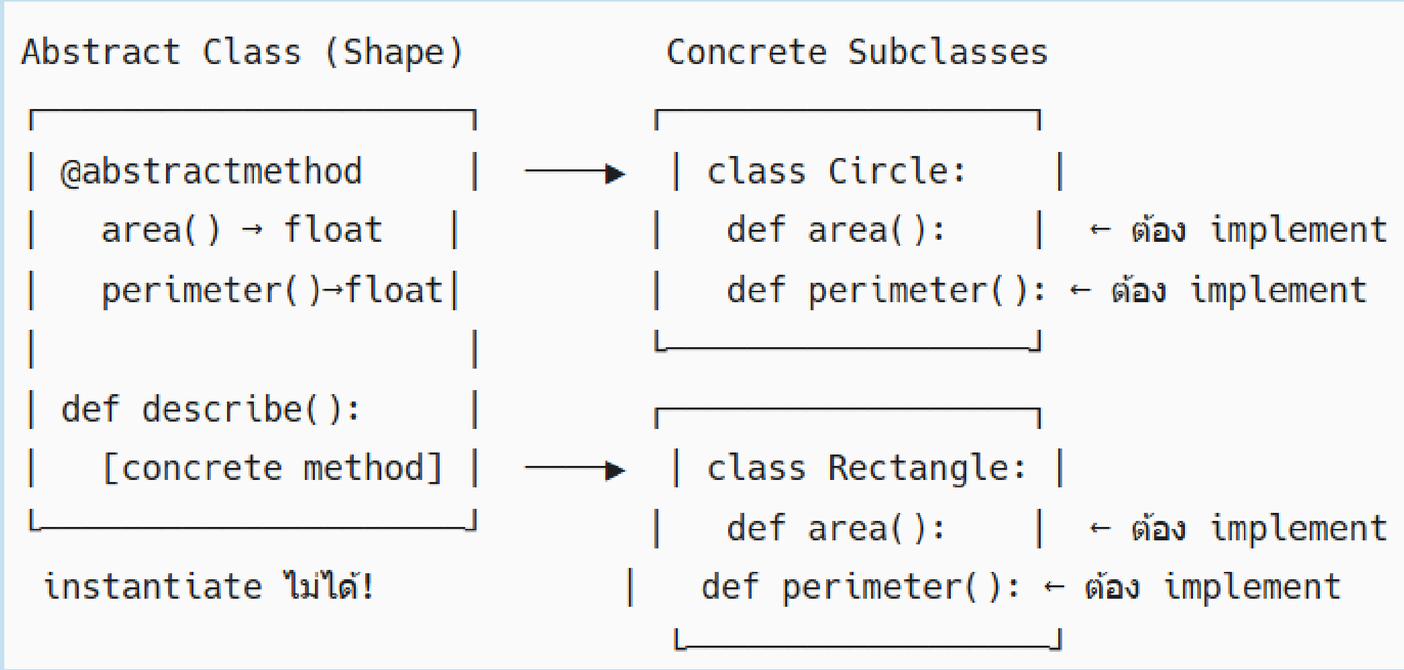
class ที่ inherit จาก ABC กลายเป็น abstract class ซึ่งมีคุณสมบัติ 2 อย่าง ได้แก่

1. instantiate ตรงๆ ไม่ได้
2. กำหนด abstract methods ที่ subclass ต้อง implement ครบ

Abstract Base Classes (ABC)

```python
from abc import ABC, abstractmethod

class Shape(ABC):    # ← inherit จาก ABC
    ...
```



class ที่ inherit จาก ABC กลายเป็น abstract class ซึ่งมี 2 คุณสมบัติหลัก ได้แก่ instantiate ตรงๆ ไม่ได้ และ กำหนด abstract methods ที่ subclass ต้อง implement ครบ

@abstractmethod

@abstractmethod

- @abstractmethod เป็น decorator ที่ทำเครื่องหมายว่า method นั้น ต้อง ถูก override ใน subclass

```python
class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        ...   # ← "..." (Ellipsis) เป็น body placeholder ของ abstract method
              #    เทียบเท่า pass แต่เป็น convention ที่ใช้กับ abstract methods

    @abstractmethod
    def perimeter(self) -> float:
        ...

    def describe(self) -> str:   # ← concrete method ใน abstract class ได้!
        return (
            f"{self.__class__.__name__}: "
            f"พื้นที่ = {self.area():.2f} หน่วย² | "
            f"เส้นรอบวง = {self.perimeter():.2f} หน่วย"
        )
```

- สังเกตว่า describe() เป็น concrete method ที่อยู่ใน abstract class และเรียก self.area() กับ self.perimeter() ซึ่งยังไม่มี implementation ณ จุดนั้น
- แต่ Python ไม่ error เพราะรู้ว่า เมื่อ describe() ถูกเรียกในชีวิตจริง self จะต้องเป็น concrete subclass ที่ implement ทั้งสอง method แล้ว

@abstractmethod

- เมื่อใส่ @abstractmethod ไว้ใน class ใด class นั้นจะกลายเป็น Abstract Class โดยอัตโนมัติ และทำหน้าที่เป็นสัญญาที่บังคับว่าทุก subclass ที่ inherit ไปต้อง implement method นั้นด้วยตัวเองอย่างครบถ้วน
- @abstractmethod ใส่ได้เฉพาะใน class ที่ประกาศสืบทอดจาก ABC เท่านั้น และใส่เหนือ method ที่ต้องการบังคับให้ subclass implement
  - ตัว method มักเขียน body เป็น pass หรือ ... เพราะไม่มี logic จริงในระดับนี้

```python
from abc import ABC, abstractmethod

class Shape(ABC):              # ต้อง inherit จาก ABC ก่อนเสมอ

    @abstractmethod            # ใส่เหนือ method ที่ต้องการบังคับ
    def area(self) -> float:
        pass                   # ไม่มี logic จริง — subclass ต้องเขียนเอง

    @abstractmethod
    def perimeter(self) -> float:
        pass

    def describe(self) -> str:           # method นี้ไม่มี @abstractmethod
        return f"Area: {self.area():.2f}"    # จึงเป็น concrete method ที่ใช้ได้เลย
```

Abstract Class สามารถมีทั้ง abstract method และ concrete method อยู่ร่วมกันได้

concrete method คือ logic ที่ทุก subclass ใช้ร่วมกันได้โดยไม่ต้องเขียนซ้ำ

ผลที่เกิดขึ้นเมื่อใส่ @abstractmethod

Abstract Class ไม่สามารถสร้าง object ได้โดยตรง Python จะ raise TypeError ทันทีหากพยายามทำเช่นนั้น

- เพราะ class ที่มี abstract method ยังไม่สมบูรณ์พอที่จะมีตัวตนได้

```
s = Shape()
```

TypeError: Can't instantiate abstract class Shape # with abstract methods area, perimeter

subclass ที่ implement ไม่ครบไม่สามารถสร้าง object ได้เช่นกัน

- หาก subclass ลืม implement abstract method ใด method หนึ่ง Python จะตรวจพบและแจ้ง error ทันทีตอนสร้าง object ไม่ใช่รอให้พังระหว่างรันโปรแกรม ซึ่งช่วยให้ตรวจพบข้อผิดพลาดได้เร็วกว่ามาก

```python
class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:              # implement แล้ว
        return 3.14159 * self.radius ** 2

    # ลืม implement perimeter() !

c = Circle(5)
```

TypeError: Can't instantiate abstract class Circle with abstract method perimeter

ผลที่เกิดขึ้นเมื่อใส่ @abstractmethod

เมื่อ implement ครบถ้วนแล้ว subclass จึงจะสร้าง object ได้ตามปกติ

```python
class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * 3.14159 * self.radius

c = Circle(5)
print(c.area())
print(c.describe())
```

implement ครบแล้ว

สร้างได้ทันที
78.53975
Area: 78.54 ← ใช้ concrete method จาก Shape

isinstance() ıla issubclass()

isinstance()

- isinstance(object, class) คือ built-in function
  - คืนค่า True หาก object นั้นเป็น instance ของ class ที่ระบุ หรือเป็น instance ของ subclass ใดๆ ที่สืบทอดมาจาก class นั้น
  - คืนค่า False ในกรณีอื่น

```
class Animal: pass
class Dog(Animal): pass
class Cat(Animal): pass

d = Dog()

print(isinstance(d, Dog))
print(isinstance(d, Animal))
print(isinstance(d, Cat))
```

True — d เป็น Dog โดยตรง

True — d เป็น Animal ด้วย เพราะ Dog inherit มา

False — d ไม่ใช่ Cat

isinstance() ไม่ได้ตรวจเฉพาะประเภทตรงๆ แต่ตรวจทั้ง hierarchy
ดังนั้น isinstance(d, Animal) จึงได้ True แม้ว่า d จะถูกสร้างจาก Dog ไม่ใช่ Animal โดยตรง

isinstance()

สามารถตรวจหลาย class พร้อมกันได้โดยส่งเป็น tuple ซึ่งจะคืนค่า True หาก object เป็นประเภทใดประเภทหนึ่งใน tuple นั้น

```
print(isinstance(d, (Dog, Cat)))
```

True — d เป็น Dog ซึ่งอยู่ใน tuple

isinstance() เหมาะสำหรับการตรวจสอบประเภทก่อนดำเนินการที่อาจเกิด error
เช่น การตรวจสอบ input ที่รับเข้ามาใน method หรือการแยก logic ในกรณีที่จำเป็นต้องจัดการต่างกันจริงๆ

```
def deposit(self, amount) -> None:
    if not isinstance(amount, (int, float)):
        raise TypeError("จำนวนเงินต้องเป็นตัวเลขเท่านั้น")
    if amount <= 0:
        raise ValueError("จำนวนเงินต้องมากกว่า 0")
    self.__balance += amount
```

issubclass()

- issubclass(child, parent) คือ built-in function ที่ตรวจสอบความสัมพันธ์ระหว่าง class สองตัว
  - โดยคืนค่า True หาก child เป็น subclass ของ parent ไม่ว่าจะเป็นทางตรงหรือผ่านการสืบทอดหลายระดับ
- สิ่งที่แตกต่างจาก isinstance() อย่างชัดเจนคือ issubclass() ทำงานกับ class เท่านั้น ไม่ใช่ object

```
print(issubclass(Circle, Shape))      # True
print(issubclass(Shape, ABC))         # True
print(issubclass(Rectangle, Circle))  # False
```

```
print(issubclass(Dog, Animal))
print(issubclass(Cat, Animal))
print(issubclass(Dog, Cat))
print(issubclass(Dog, Dog))
```

True — Dog สืบทอดจาก Animal

True — Cat สืบทอดจาก Animal

False — Dog ไม่ได้สืบทอดจาก Cat

True — class ถือว่าเป็น subclass ของตัวเอง

issubclass()

- เหมาะสำหรับการตรวจสอบโครงสร้างของ class hierarchy ในระดับ meta
- เช่น การเขียน framework หรือ utility function ที่ต้องตรวจสอบว่า class ที่รับเข้ามามีความสัมพันธ์กับ class ที่กำหนดหรือไม่

```
def register_payment_method(cls) -> None:
    if not issubclass(cls, Payment):
        raise TypeError(f"{cls.__name__} ต้องสืบทอดจาก Payment")
```

ข้อควรระวัง

- ห้ามนำ isinstance() ไปใช้แทนที่ Polymorphism

```
def process(shape):
    if isinstance(shape, Circle):
        return 3.14 * shape.radius ** 2
    elif isinstance(shape, Rectangle):
        return shape.width * shape.height
```

ทุกครั้งที่เพิ่ม class ใหม่ต้องกลับมาแก้ไข function นี้เสมอ

วิธีที่ถูกต้องคือออกแบบให้ทุก class มี interface ร่วมและเรียกใช้ผ่าน Polymorphism แทน

แนะนำ — ใช้ Polymorphism

```
def process(shape: Shape):
    return shape.area()
```

ไม่สนใจประเภท เรียก interface ร่วมได้เลย

ฝึกเขียนโปรแกรม



Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 1 – วิเคราะห์ก่อนเขียนโค้ด: เมื่อไรควรใช้ Abstract Class

- ตอบคำถาม 3 ข้อนี้ก่อนเสมอ เพื่อนำไปสู่การออกแบบ Abstract Class ที่ถูกต้อง

  1) มี "กลุ่มของสิ่งที่คล้ายกัน" หรือไม่?
    - Circle และ Rectangle ต่างก็เป็น "รูปทรง" (Shape) เหมือนกัน

  2) ทุกตัวใน "กลุ่ม" นั้นทำสิ่งเดียวกันได้ แต่ด้วยวิธีต่างกันหรือไม่?
    - Circle และ Rectangle ต้องคำนวณพื้นที่ (area()) และ เส้นรอบรูปได้ (perimeter()) เหมือนกัน
    - แต่ สูตรการคำนวณต่างกันโดยสิ้นเชิง วงกลมใช้ $\pi \times r^2$ ส่วนสี่เหลี่ยมใช้ width × height
    - นี่คือสัญญาณสำคัญที่บอกว่าควรใช้ Abstract Method

  3) การสร้าง object จาก parent โดยตรง "สมเหตุสมผล" หรือไม่?
    - "รูปทรง" ที่ไม่ระบุว่าเป็นวงกลมหรือสี่เหลี่ยม มีพื้นที่เท่าไร คำตอบคือตอบไม่ได้เลย เพราะ "Shape" เฉยๆ ไม่มีสูตรคำนวณของตัวเอง ดังนั้น Shape() ไม่ควรสร้าง object ได้
    - มันจึงต้องเป็น Abstract Class ไม่ใช่ class ธรรมดา

- เมื่อคำตอบของทั้งสามข้อสอดคล้องกัน (มีกลุ่ม, ทำสิ่งเดียวกันแต่ต่างวิธี, สร้าง object จาก parent ตรงๆ ไม่สมเหตุสมผล) จึงสรุปได้ว่า Abstract Class คือคำตอบที่เหมาะสม

Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 2 – วางโครงสร้างก่อนเขียนโค้ด

- แบ่งสิ่งที่ทุก subclass ต้องมีออกเป็นสองประเภท
  - Abstract Methods (ต้อง implement เอง ทุกตัว สูตรต่างกัน)
    - ได้แก่ area() ที่คืนค่าพื้นที่ และ perimeter() ที่คืนค่าเส้นรอบรูป
  - Concrete Methods (ใช้ร่วมกันได้ทันที ไม่ต้องเขียนซ้ำ)
    - ได้แก่ describe() ที่แสดงข้อมูลรูปทรงในรูปแบบเดียวกันสำหรับทุก subclass โดยเรียกใช้ self.area() และ self.perimeter()
- describe() เขียนเพียงครั้งเดียวใน Shape แต่ใช้ได้กับทุก subclass เพราะเมื่อถูกเรียกจริง Python รู้ว่า self คือ object ของ subclass ที่ implement area() และ perimeter() ไว้แล้วเสมอ

Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 3 – Shape: Abstract Class ที่ต้องเขียน

```
from abc import ABC, abstractmethod

class Shape(ABC):
```

- Import ที่ต้องมี คือ from abc import ABC, abstractmethod
  - ABC คือ class แม่พิเศษที่ทำให้ subclass ไม่สามารถสร้าง object ได้หากยังมี abstract method ที่ไม่ได้ implement
  - abstractmethod คือ decorator ที่ใช้ประกาศ method ที่บังคับให้ subclass implement
- Shape ไม่มี _init_ ของตัวเอง ในโจทย์นี้ เพราะ Shape ไม่มี attribute ร่วมที่ทุกรูปทรงต้องมี
  - วงกลมมี radius, สี่เหลี่ยมมี width, height (ไม่เหมือนกันเลย)
  - หากทุก subclass มี attribute ร่วมกันบางตัว เช่น color ค่อยเพิ่ม _init_ ใน Shape ภายหลัง

Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 4 – Abstract Methods ที่ต้องเขียนใน Shape

```
@abstractmethod
def area(self)
    pass
```

- เขียนเหนือ method ด้วย @abstractmethod เสมอ
- body ของ method เขียนเพียง pass เพราะไม่มี logic จริงในระดับนี้
- area() ประกาศไว้เฉยๆ ไม่ต้องมี logic ใดๆ เป็นเพียงสัญญาว่า "subclass ทุกตัวต้องมี method ชื่อนี้ รับแค่ self และคืนค่าเป็น float"

```
@abstractmethod
def perimeter(self)
    pass
```

perimeter() เขียนในรูปแบบเดียวกัน เป็นสัญญาว่า subclass ต้อง implement เช่นกัน

Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 5 – Concrete Method ที่ต้องเขียนใน Shape

```
def describe(self)
```

- describe() ไม่มี @abstractmethod เพราะมันมี logic จริงที่ใช้ร่วมกันได้ทุก subclass
- เขียนเพียงครั้งเดียว โดยเรียกใช้ self.area() และ self.perimeter()
- แม้ว่า Shape.area() เองจะไม่มี logic (มีแค่ pass) แต่ self.area() ที่ถูกเรียกจริงจะวิ่งไปหา method ของ subclass เสมอ (Circle หรือ Rectangle)
- รูปแบบข้อความที่ต้องแสดงคือ "ชื่อคลาส | พื้นที่: X.XX | เส้นรอบรูป: Y.YY"
    - โดยใช้ type(self).__name__ เพื่อดึงชื่อ class ของ object จริง (ไม่ใช่เขียนชื่อตายตัว เพราะ describe() ต้องใช้ได้กับทุก subclass)

เฉลย Shape

```python
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):
        print(f"{type(self).__name__} | พื้นที่: {self.area():.2f} | เส้นรอบรูป: {self.perimeter():.2f}")
```

Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 6 – Circle ที่ต้องเขียน

```
class Circle(Shape):
    def __init__(self, radius: float):
```

- Circle ต้อง inherit จาก Shape เสมอ เพราะต้องรับ abstract methods ที่ Shape ประกาศไว้มา implement
- Attribute ที่ต้องมี คือ _radius เก็บเป็น float มีช่วงค่าที่ถูกต้องคือมากกว่า 0 เท่านั้น
  - ให้ทำเป็น property + setter ตามหลัก Encapsulation ที่เรียนมา (ตรวจ type ก่อน แล้วตรวจค่า)
- Method ที่ต้อง implement คือ
  - area() ที่คืนค่า math.pi * self.radius ** 2
  - perimeter() ที่คืนค่า 2 * math.pi * self.radius
  - ทั้งสอง method ต้องไม่มี @abstractmethod เพราะเป็นการ implement จริงแล้ว

เฉลย Circle

```python
import math
from shape import Shape

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, new_r):
        if not isinstance(new_r, (int,float)):
            raise TypeError("รัศมีควรเป็นตัวเลข")
        if new_r <= 0:
            raise ValueError("ค่ารัศมีต้องมากกว่า 0")
        self.__radius = new_r

    def area(self):
        return math.pi * self.radius **2

    def perimeter(self):
        return 2 * math.pi * self.radius
```

Live Coding: Shape (ABC) → Circle, Rectangle

ส่วนที่ 7 – Rectangle ที่ต้องเขียน

```
class Rectangle(Shape):
    def __init__(self, width: float, height: float):
```

- Attribute ที่ต้องมี คือ _width และ _height
  - เก็บเป็น float ทั้งคู่ มีช่วงค่าที่ถูกต้องคือมากกว่า 0 เท่านั้น
  - ทำเป็น property + setter เช่นเดียวกับ Circle
- Method ที่ต้อง implement คือ
  - area() ที่คืนค่า self.width * self.height
  - perimeter() ที่คืนค่า 2 * (self.width + self.height)

เฉลย Rectangle

```python
from shape import Shape

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def width(self):
        return self.__width

    @width.setter
    def width(self, value):
        if not isinstance(value, (int,float)):
            raise TypeError("ความกว้างต้องเป็นตัวเลข")
        if value <= 0:
            raise ValueError("ความกว้างต้องมากกว่า 0")
        self.__width = value
```

```python
    @property
    def height(self):
        return self.__height

    @height.setter
    def height(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("ความสูงต้องเป็นตัวเลข")
        if value <= 0:
            raise ValueError("ความสูงต้องมากกว่า 0")
        self.__height = value

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)
```

ทดสอบการทำงาน

```
5    # ทดสอบว่า Shape สร้าง object ตรงๆ ไม่ได้
6    try:
7        s = Shape()
8    except TypeError as e:
9        print(f"แจ้งเตือน: {e}")
```

```
แจ้งเตือน: Can't instantiate abstract class Shape without an implementation for abstract methods 'area', 'perimeter'
```

ทดสอบการทำงาน

```
c = Circle(5)
r = Rectangle(4, 6)

c.describe()
r.describe()
```

```
Circle | พื้นที่: 78.54 | เส้นรอบรูป: 31.42
Rectangle | พื้นที่: 24.00 | เส้นรอบรูป: 20.00
```

```
shapes = [Circle(3), Rectangle(5, 8), Circle(10)]
for shape in shapes:
    shape.describe()
```

```
Circle | พื้นที่: 28.27 | เส้นรอบรูป: 18.85
Rectangle | พื้นที่: 40.00 | เส้นรอบรูป: 26.00
Circle | พื้นที่: 314.16 | เส้นรอบรูป: 62.83
```

ทดสอบการทำงาน

```
print(isinstance(c, Shape))
print(issubclass(Circle, Shape))
```

```
True
True
```

แจ้งเตือน: ค่ารัศมีต้องมากกว่า 0

```
try:
    bad_circle = Circle(-5)
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
```

แจ้งเตือน: ความสูงต้องมากกว่า 0

```
try:
    bad_rect = Rectangle(4, -6)
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
```

แจ้งเตือน: รัศมีควรเป็นตัวเลข

```
try:
    bad_circle2 = Circle("ห้า")
except TypeError as e:
    print(f"แจ้งเตือน: {e}")
```

แจ้งเตือน: ความกว้างต้องมากกว่า 0

```
try:
    bad_rect2 = Rectangle(-4, 6)
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
```

Common Mistakes

ผิดพลาดที่ 1: พยายาม instantiate abstract class โดยตรง

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

s = Shape()
```

TypeError ทันที

Abstract class **มีไว้เป็น** template **เท่านั้น ต้องสร้าง** concrete subclass **ที่** implement **ทุก** abstract method **ก่อน**

ผิดพลาดที่ 2: ลืม implement abstract method บางตัว

```
class Triangle(Shape):
    def __init__(self, base: float, height: float):
        self.base   = base
        self.height = height

    def area(self) -> float:
        return 0.5 * self.base * self.height
    # ลืม perimeter()!

t = Triangle(3.0, 4.0)
```

TypeError: Can't instantiate abstract class Triangle without an implementation for abstract method 'perimeter'

วิธีตรวจสอบ: ดู abstract methods **ทั้งหมดใน** parent **ด้วย** Shape.__abstractmethods__

ผิดพลาดที่ 3: ใช้ isinstance() chain แทน polymorphism

ผิด — code นี้ต้องแก้ทุกครั้งที่เพิ่ม shape ใหม่

```
def total_area(shapes):
    total = 0
    for s in shapes:
        if isinstance(s, Circle):
            total += math.pi * s.radius ** 2
        elif isinstance(s, Rectangle):
            total += s.width * s.height
    return total
```

ถูก — polymorphic, รองรับ shape ใหม่อัตโนมัติ

```
def total_area(shapes):
    return sum(s.area() for s in shapes)
```

ผิดพลาดที่ 4: ลืม import ABC หรือ abstractmethod

ผิด — ไม่ import

```
class Shape(ABC):
    @abstractmethod
    def area(self): ...
```

NameError: name 'ABC' is not defined

NameError: name 'abstractmethod' is not defined

ถูก

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
```

ผิดพลาดที่ 5: Abstract method มี body แต่ subclass คิดว่าไม่ต้อง implement

```python
class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        return 0.0
```

← Python อนุญาตให้มี body ใน abstract method ได้ แต่ subclass ยังต้อง override อยู่เหมือนเดิม
body นี้เรียกได้ผ่าน super().area() เท่านั้น

```python
class Circle(Shape):
    pass
```

← ยังได้ TypeError เมื่อ instantiate แม้ parent มี body